Packaging software for the Debian based systems

One of the most important things that you need to learn is to package and publish the code to make it available for the end users. Lets try and do this in a step by step manner as part of our simple tutorial.

Before you begin with creating packages, make sure that the following packages are installed on your system to enable you to use the utilities related to creating a deb package: -

- build-essential
- devscripts
- debhelper
- lintian

In order to understand the concept of creating packages, we need to learn that creating a package is mainly a three step process: -

- *Procuring the source tarball :* The source tarball is the compressed tar file of the source code of the application to be packaged, eg. packagename.tar.gz
- *Creating the Source package :* This step involves adding Debian specific packaging files and any modifications that you want to make to the package. This is done under a special directory named 'debian'.
- *Building the package:* Once the source package is ready, it can be built to create a binary package which can then be finally installed on the system.

In the first step, the source tarball needs to be renamed speciafically according to the accepted standards, mainly the fact that it needs to in lowercase characters and in the format like: -

```
mv packagename-1.0.0.tar,gz packagename_1.0.0.orig.tar.gz
```

We can now unpack the source tarball and create the debian subdirectory under the source tree. Inside this directory following files are necessary to be provided: -

- *changelog :* This file summrizes the list of changes in this particular version of the package as compared to the previous version or mention that it is an initial release in case it is being packaged for the first time. This file can be easily created using the dch command as follows: -

```
dch -create -v 1.0.0 --package packagename
```

Dpkg showing metadata about a package in Ubuntu

This will create a changelog file with a format as follows: -

```
vlc (2.0.8) UNRELEASED; urgency=low
  * Initial release. (Closes: #XXXXXX)

 -- Ankit Mathur <ankit@ankit-laptop>  Sun, 15 Dec 2013 19:10:31 +0530
```

- *compat :* This file specifies the compatibility level of the application for debhelper, which contains a single digit number like 9 or 8 as its contents.
- *control :* This file provides information about the source and binary package, mainly details about the maintainer, priority, section, dependencies, description etc. A sample control file looks as follows: -

```
Source: helloworld
Section: unknown
Priority: extra
Maintainer: Ankit Mathur <ankit@unknown>
Build-Depends: debhelper (>= 8.0.0)
Standards-Version: 3.9.4
Homepage: <insert the upstream URL, if relevant>
#Vcs-Git: git://git.debian.org/collab-maint/helloworld.git
#Vcs-Browser: http://git.debian.org/?p=collab-maint/helloworld.git;a=summary
Package: helloworld
Architecture: any
Depends: ${shlibs:Depends}, ${misc:Depends}
Description: <insert up to 60 chars description>
<insert long description, indented with spaces>
```

- *copyright :* This file keeps track of legal and copyright information.
- *rules :* This file acts as the makefile and consists of rules each of which specifies a target and how it is executed. This target specifies the various build targets which can be overriden when executed in the debhelper sequence.The most basic file looks like this: -

```
#!/usr/bin/make -f
# -*- makefile -*-
# Sample debian/rules that uses debhelper.
# This file was originally written by Joey Hess and Craig Small.
# As a special exception, when this file is copied by dh-make into a
# dh-make output file, you may use that output file without restriction.
# This special exception was added by Craig Small in version 0.37 of dh-make.


# Uncomment this to turn on verbose mode.
#export DH_VERBOSE=1


%:
        dh $@
```

- *source/format :* This file contains the version information about the format of the source package, for eg. 3.0 (quilt) in this case.

There are a couple of other files that might be needded for completing the package and most of this process involves writing rules and ensuring that build process works correctly as expected, but once all that is done, the package can be built as follows: -

```
debuild
```